Formal Problem Definition and Maximum-Cardinality Baseline

CS3212
Design and Analysis of Algorithm

Course Project

Fair and Capacity-Constrained Bipartite Matching

Contents

1. Problem Overview....3
   1.1 Approach....3
   1.2 Objectives....3
2. Mathematical Model....3
3. Baseline Constraints....4
4. Baseline Objective....4
5. Input and Output....5
   5.1 Input....5
   5.2 Output....5
6. Maximum-Cardinality Algorithm....5
7. Pseudocode....6
8. Correctness Argument....7
9. Time and Space Complexity....7
10. Example and tests....7

1. Problem Overview

1.1 Approach

i) Bipartite matching is a graph-based approach. We can assign elements of one set to elements of another set, satisfying some conditions.
ii) In our project, the two sets are **Agents** and **Resources.**
iii) An assignment is only possible when corresponding agent-resource pair is eligible.
iv) Current initial stage of the project considers the **maximum-cardinality matching** problem.

1.2 Objectives

a. To assign the maximum possible number of agents to eligible resources.
b. To ensure that each agent is assigned to at most one resource.
c. To ensure, in baseline case, each resource is assigned to at most one agent.

This baseline provides the foundation for the later stages of the project, where utility, resource capacities, and fairness considerations will be introduced.

2. Mathematical Model

2.1 Let the set of agents be represented by:

$$U = \{u_1, u_2, \dots, u_n\}$$

2.2 Let the set of resources be represented by:

$$V = \{v_1, v_2, \dots, v_m\}$$

2.3 The problem is represented as a bipartite graph:

$$G = (U, V, E)$$

where, E represents the set of possible connections between agents and resources.

2.4 An agent u_i can be assigned to a resource v_j only when the corresponding pair belongs to E:

$$(u_i, v_j) \in E$$

Here, i represents an agent index and j represents a resource index.

2.5 For each possible assignment, define a binary decision variable:

$$x_{ij} = \begin{cases} 1, & \text{if agent } u_i \text{ is assigned to resource } v_j \\ 0, & \text{otherwise} \end{cases}$$

2.6 For the baseline stage, every resource has capacity:

$$c_j = 1$$

3. Baseline Constraints

The baseline matching must satisfy the following constraints:

3.1 At most one resource per agent

Each agent can be assigned to at most one resource:

$$\sum_{j=1}^{m} x_{ij} \leq 1 \qquad \forall\, i \in \{1,2,\dots,n\}$$

3.2 At most one agent per resource

Since the baseline uses capacity 1, each resource can be assigned to at most one agent:

$$\sum_{i=1}^{n} x_{ij} \leq 1 \qquad \forall\, j \in \{1,2,\dots,m\}$$

3.3 Binary assignment

Each assignment variable can only take the values 0 or 1:

$$x_{ij} \in \{0,1\}$$

4. Baseline Objective

As mentioned in 1.2 (a), the objective of the baseline stage is to find a valid matching that assigns the **maximum possible number of agents** to resources.

Let x_{ij} indicate whether agent u_i is assigned to resource v_j.
The objective is to maximize the total number of selected assignments.

$$\max \sum_{i=1}^{n} \sum_{j=1}^{m} x_{ij}$$

Each selected assignment contributes 1 to the total. Therefore, maximizing this value gives the **maximum-cardinality matching**, that is, the largest possible number of matched agent-resource pairs while satisfying the baseline constraints.

Ex., if 5 agents are present but only 3 can be validly assigned, the maximum-cardinality matching will contain 3 matched pairs. The remaining 2 agents will remain unmatched.

5. Input and Output

5.1 Input

The baseline algorithm takes a bipartite graph consisting of agents, resources, and their eligible connections. The input follows the common project data structure agreed upon by the team.

For the baseline stage, the important information is:

i) Agents: A set of agents to be assigned.
ii) Resources: A set of available resources.
iii) Eligibility: An agent-resource pair is included only when that assignment is allowed.
iv) Capacity: Each resource has a capacity field. For the maximum-cardinality baseline, the capacity of every resource is set to 1.

The baseline does not use utility or fairness information for deciding the matching. These will be introduced in later stages.

5.2 Output

The algorithm returns:

i) The list of matched agent-resource pairs.
ii) The total number of matched pairs.
iii) Agents that could not be assigned, if any.

6. Maximum-Cardinality Algorithm

For the baseline, we use a **bipartite matching algorithm based on augmenting paths**. The algorithm starts with no assignments and tries to find a new valid assignment for each agent.

For every agent, it checks its eligible resources. If a free resource is found, the agent is assigned to it. If all eligible resources are already occupied, the algorithm tries to rearrange an existing assignment so that the current agent can also be matched. This process is repeated until no further assignment can be made.

The resulting matching contains the **maximum possible number of valid agent-resource pairs**.

The core algorithm will be implemented directly in **C++**, without using a black-box matching function or external solver, as required by the project plan.

7. Pseudocode

i) The baseline uses an **augmenting-path based bipartite matching approach**.
ii) For each agent, the algorithm tries to find a free resource.
iii) If an eligible resource is already assigned, it recursively tries to move the previously assigned agent to another resource.

```
MAXIMUM_CARDINALITY_MATCHING(G = (U, V, E))

1. Initialize match[v] = NIL for every resource v in V.
2. matchingSize = 0

3. For each agent u in U:
      Mark all resources as unvisited.
      If FIND_AUGMENTING_PATH(u) is true:
         matchingSize = matchingSize + 1

4. Return all matched agent-resource pairs and matchingSize


FIND_AUGMENTING_PATH(u)

1. For each resource v adjacent to u:
      If v is not visited:
         Mark v as visited.

         If match[v] = NIL:
            match[v] = u
            Return true

         Else if FIND_AUGMENTING_PATH(match[v]) is true:
            match[v] = u
            Return true

2. Return false
```

8. Correctness Argument

The algorithm maintains a **valid matching** throughout its execution because an agent is assigned to at most one resource and a resource stores at most one matched agent.

Whenever FIND_AUGMENTING_PATH succeeds, one previously unmatched agent is connected to a free resource through an alternating path. The reassignment along this path removes conflicts and increases the size of the matching by exactly one.

After all agents have been processed, if no augmenting path can be found from any unmatched agent, the matching is maximum. According to the **augmenting-path characterization of maximum matching**, a matching is maximum when no augmenting path exists.

Therefore, the algorithm returns a **maximum-cardinality bipartite matching**.

9. Time and Space Complexity

Let:

$$\begin{gathered} E = \text{number of eligible agent-resource edges} \\ n = \text{number of agents} \\ m = \text{number of resources} \end{gathered}$$

For the DFS-based augmenting-path implementation, each agent may perform a search over the graph.

i) **Worst-case time complexity:** $O(nE)$
ii) **Auxiliary space complexity:** $O(n + m + E)$

The space is used for the graph representation, the current matching, and the visited information used during each augmenting-path search.

10. Example and tests

Consider:

$$\text{Agents:} \quad U = \{u1, u2, u3\}$$

$$\text{Resources:} \quad V = \{v1, v2, v3\}$$

Eligible assignments:

$$\begin{gathered} u1 \rightarrow v1, v2 \\ u2 \rightarrow v1 \\ u3 \rightarrow v2, v3 \end{gathered}$$

One maximum matching is:

$$\begin{gathered} u1 \rightarrow v2 \\ u2 \rightarrow v1 \\ u3 \rightarrow v3 \end{gathered}$$

Therefore, the matching contains **3 pairs**, so all three agents are assigned: $|M| = 3$

Test Cases

TEST CASE	INPUT CONDITION	EXPECTED RESULT
1	Empty graph	0 matched agents
2	No eligible edges	0 matched agents
3	More agents than resources	At most the number of available resources can be matched
4	Complete bipartite graph with equal set sizes	All agents matched
5	Conflicting assignments	Algorithm rearranges existing matches when possible